

Student Name: KAMAGAJU ANITHA

Student ID: 28608

Course: Database Development with PL/SQL

DATE :25/02/2026

lecture:

ASSIGNMENT 3

Q1.1 Declare student name, GPA, date, and honor roll variables. Print formatted output.

```
-- =====  
-- QUESTION 1 — Variables & DBMS_OUTPUT  
-- Student: Kamagaju anitha | ID: 28608  
-- =====
```

SET SERVEROUTPUT ON;

DECLARE

```
  v_full_name VARCHAR2(50) := 'Bob smith'; -- student full name  
  v_gpa      NUMBER := 3.2;                -- GPA between 0 and 4  
  v_today    DATE := SYSDATE;              -- today's date  
  v_honor    BOOLEAN;                      -- TRUE if GPA > 3.5
```

BEGIN

```
-- Assign boolean based on GPA
```

```
v_honor := (v_gpa > 3.5);
```

```
-- Print results
```

```
DBMS_OUTPUT.PUT_LINE('Student: ' || v_full_name);
```

```
DBMS_OUTPUT.PUT_LINE('GPA: ' || v_gpa);
```

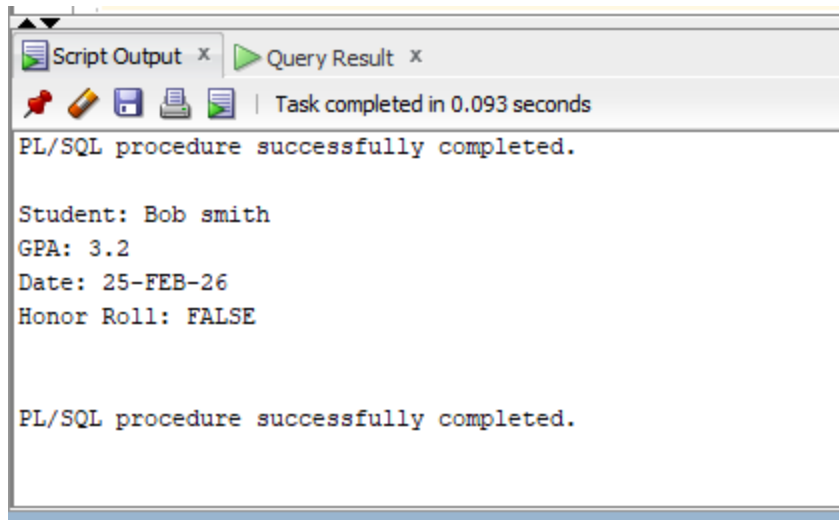
```
DBMS_OUTPUT.PUT_LINE('Date: ' || TO_CHAR(v_today,'DD-MON-YY'));
```

```
DBMS_OUTPUT.PUT_LINE('Honor Roll: ' || CASE WHEN v_honor THEN 'TRUE' ELSE  
'FALSE' END);
```

```
END;
```

```
/
```

Output screenshot



QUESTION 1.2 — Fetch student info using SELECT INTO

SET SERVEROUTPUT ON

DECLARE

 v_first_name VARCHAR2(50); -- assuming first_name column is VARCHAR2(50)

 v_last_name VARCHAR2(50); -- assuming last_name column is VARCHAR2(50)

 v_gpa NUMBER(3,2); -- GPA usually stored as NUMBER(3,2), e.g., 3.80

BEGIN

 -- Fetch student_id = 1005

 SELECT first_name, last_name, gpa

 INTO v_first_name, v_last_name, v_gpa

 FROM students

 WHERE student_id = 1005;

 -- Print results

 DBMS_OUTPUT.PUT_LINE('Student: ' || v_first_name || ' ' || v_last_name);

 DBMS_OUTPUT.PUT_LINE('GPA: ' || v_gpa);

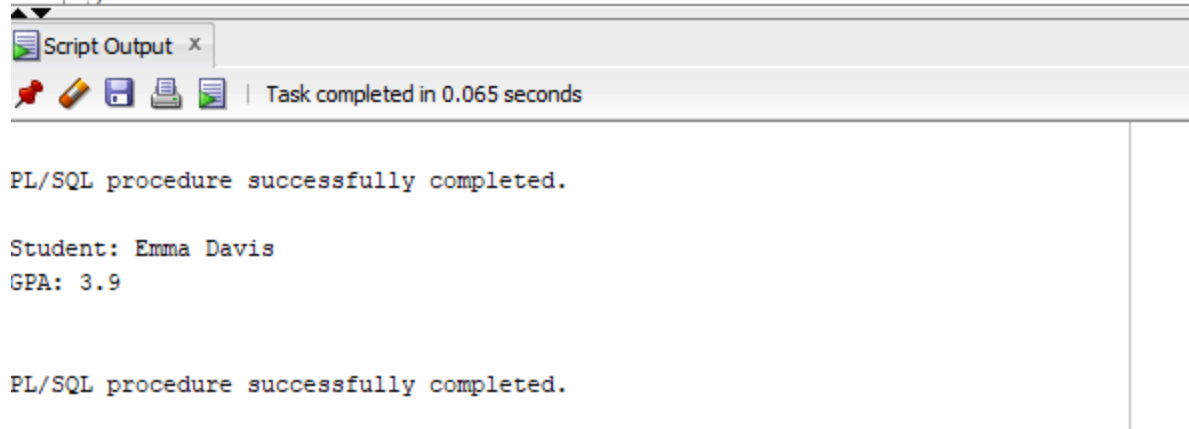
END;

/

```

-- Print results
DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || v_salary);
DBMS_OUTPUT.PUT_LINE('15% Increment: ' || v_increment);
DBMS_OUTPUT.PUT_LINE('New Salary: ' || (v_salary + v_increment));
END;
/

```



```

PL/SQL procedure successfully completed.

Student: Emma Davis
GPA: 3.9

PL/SQL procedure successfully completed.

```

Q1.3

-- question 1.3 — Employee salary increment using %TYPE

SET SERVEROUTPUT ON

DECLARE

 v_salary employees.salary%TYPE; -- automatically uses salary column datatype

 v_increment NUMBER; -- can use NUMBER since we calculate 15%

BEGIN

 -- Fetch salary of employee_id = 7

 SELECT salary

 INTO v_salary

 FROM employees

 WHERE employee_id = 7;

 -- Compute 15% increment

 v_increment := v_salary * 0.15;

 -- Print results

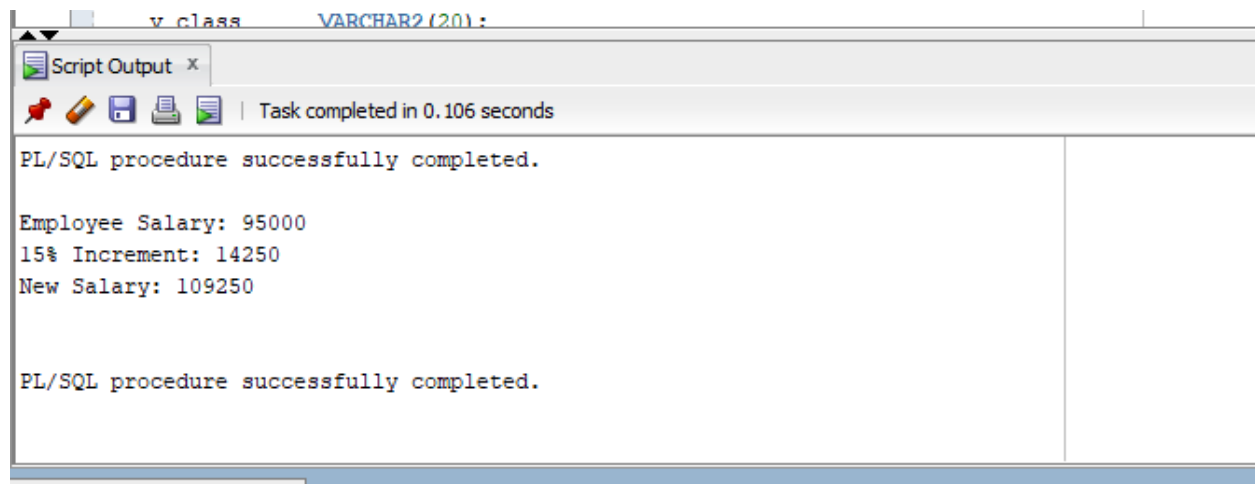
 DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || v_salary);

 DBMS_OUTPUT.PUT_LINE('15% Increment: ' || v_increment);

 DBMS_OUTPUT.PUT_LINE('New Salary: ' || (v_salary + v_increment));

END;

/



```
-- =====  
-- QUESTION 2 — IF / ELSIF / ELSE  
-- QUESTION 2.1 — GPA Classification  
-- Student: Kamagaju Anitha | ID: 28608  
-- =====
```

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
  v_first_name students.first_name%TYPE;  
  v_last_name  students.last_name%TYPE;  
  v_gpa        NUMBER(3,2); -- GPA can be 0.00 to 4.00  
  v_class      VARCHAR2(20);  
  v_status     VARCHAR2(50);
```

```
BEGIN
```

```
  -- Fetch student name and GPA  
  SELECT first_name, last_name, gpa  
    INTO v_first_name, v_last_name, v_gpa  
  FROM students  
 WHERE student_id = 1011;
```

```
  -- Classify GPA
```

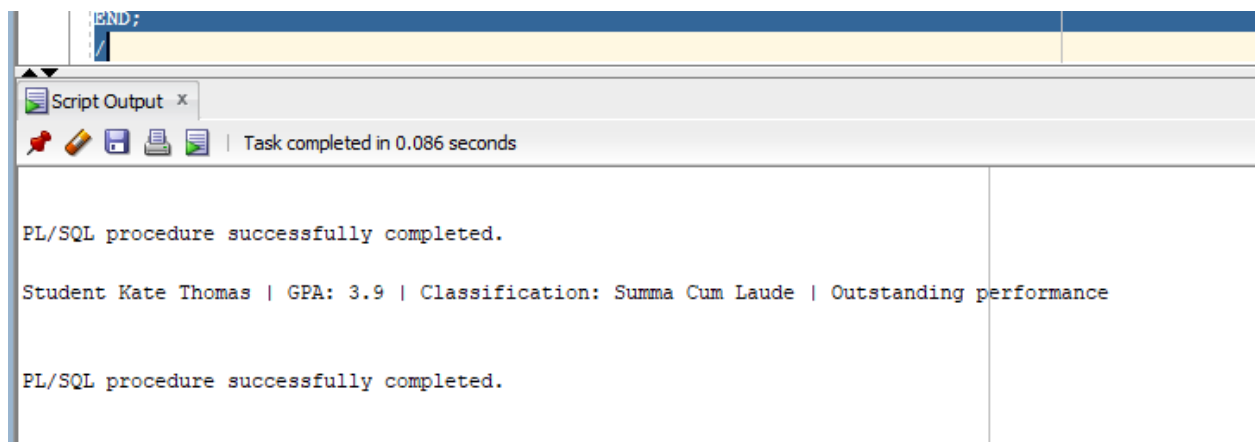
```
  IF v_gpa >= 3.7 AND v_gpa <= 4.0 THEN  
    v_class := 'Summa Cum Laude';  
    v_status := 'Outstanding performance';  
  ELSIF v_gpa >= 3.3 AND v_gpa <= 3.6 THEN  
    v_class := 'Magna Cum Laude';  
    v_status := 'Excellent performance';
```

```

ELSIF v_gpa >= 3.0 AND v_gpa <= 3.2 THEN
    v_class := 'Cum Laude';
    v_status := 'Good performance';
ELSIF v_gpa >= 2.0 AND v_gpa <= 2.9 THEN
    v_class := 'Satisfactory';
    v_status := 'Needs improvement';
ELSE
    v_class := 'Academic Probation';
    v_status := 'Urgent action required';
END IF;

-- Print the result
DBMS_OUTPUT.PUT_LINE('Student ' || v_first_name || ' ' || v_last_name ||
    ' | GPA: ' || v_gpa ||
    ' | Classification: ' || v_class ||
    ' | ' || v_status);
END;
/

```



Q2.2

SET SERVEROUTPUT ON;

DECLARE

```

v_salary employees.salary%TYPE;
v_dept employees.department%TYPE;
v_band VARCHAR2(30);
v_emp_name employees.first_name%TYPE;

```

BEGIN

```

SELECT salary, department, first_name
INTO v_salary, v_dept, v_emp_name

```

```
FROM employees
WHERE employee_id = 2;
```

```
IF v_salary < 50000 THEN
    v_band := 'Entry Level';
ELSIF v_salary BETWEEN 50000 AND 69999 THEN
    v_band := 'Mid Level';
ELSIF v_salary BETWEEN 70000 AND 89999 THEN
    v_band := 'Senior Level';
ELSE
    v_band := 'Executive Level';
END IF;
```

```
DBMS_OUTPUT.PUT_LINE('Employee: ' || v_emp_name ||
    ' | Salary: ' || v_salary ||
    ' | Department: ' || v_dept ||
    ' | Band: ' || v_band);
```

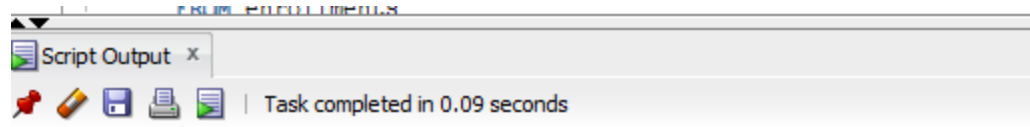
```
IF v_dept = 'Computer Science'
    AND (v_band = 'Senior Level' OR v_band = 'Executive Level') THEN
    DBMS_OUTPUT.PUT_LINE('Eligible for Tech Leadership Program');
END IF;
```

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found with ID 2.');
```

```
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Multiple employees found.');
```

```
END;
```

```
/
```



PL/SQL procedure successfully completed.

Employee: Sarah | Salary: 75000 | Department: Computer
Science | Band: Senior Level

PL/SQL procedure successfully completed.

-- QUESTION 2.3 — Enrollment Status Check

SET SERVEROUTPUT ON

DECLARE

 v_grade enrollments.grade%TYPE;

 v_score enrollments.score%TYPE;

 v_count NUMBER;

BEGIN

 -- Check if student is enrolled in course 103

 SELECT COUNT(*)

 INTO v_count

 FROM enrollments

 WHERE student_id = 1003

 AND course_id = 103;

 IF v_count = 0 THEN

 DBMS_OUTPUT.PUT_LINE('Student not enrolled in this course');

 ELSE

 -- Fetch grade and score

 SELECT grade, score

 INTO v_grade, v_score

 FROM enrollments

 WHERE student_id = 1003

 AND course_id = 103;

 IF v_grade IS NOT NULL THEN

 DBMS_OUTPUT.PUT_LINE('Grade: ' || v_grade || ' | Score: ' || v_score);

 ELSE

 DBMS_OUTPUT.PUT_LINE('Grade pending');

 END IF;

 END IF;

END;

/

```
-- =====
-- QUESTION 3 — CASE Statement
-- Task 3.1 — Grade Letter to GPA Points
-- Student: Kamagaju Anitha | ID: 28608
-- =====

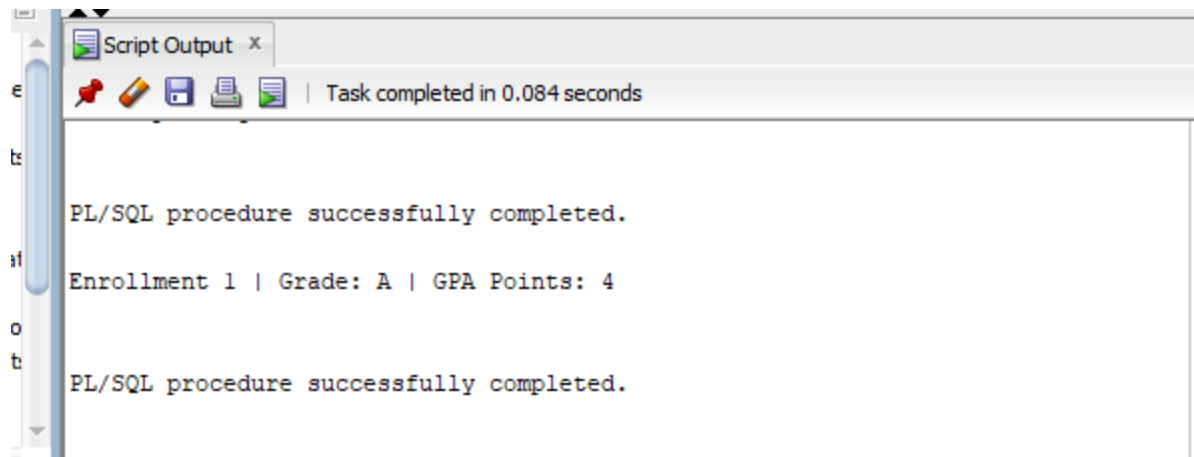
SET SERVEROUTPUT ON;

DECLARE
    v_grade enrollments.grade%TYPE;
    v_gpa   NUMBER;
BEGIN
    -- Fetch grade for enrollment_id = 1
    SELECT grade
        INTO v_grade
        FROM enrollments
        WHERE enrollment_id = 1;

    -- Convert grade to GPA points using CASE
    v_gpa := CASE v_grade
        WHEN 'A' THEN 4.0
        WHEN 'B' THEN 3.0
        WHEN 'C' THEN 2.0
        WHEN 'D' THEN 1.0
        WHEN 'F' THEN 0.0
        ELSE NULL
    END;

    -- Print results
    DBMS_OUTPUT.PUT_LINE('Enrollment 1 | Grade: ' ||
        NVL(v_grade, 'Grade not yet assigned') ||
        ' | GPA Points: ' ||
        NVL(TO_CHAR(v_gpa), 'Grade not yet assigned'));

END;
/
```

-- Task 3.2 — Department Budget Alert

SET SERVEROUTPUT ON

DECLARE

 v_dept_name departments.department_name%TYPE;

 v_budget departments.budget%TYPE;

 v_manager departments.manager_id%TYPE;

 v_class VARCHAR2(100);

 v_manager_msg VARCHAR2(50);

BEGIN

 -- Fetch one department (Computer Science)

 SELECT department_name, budget, manager_id

 INTO v_dept_name, v_budget, v_manager

 FROM departments

 WHERE department_name = 'Computer Science';

 -- Classify budget using searched CASE

 v_class := CASE

 WHEN v_budget > 700000 THEN 'High Budget — Full spending authority'

 WHEN v_budget BETWEEN 500000 AND 700000 THEN 'Medium Budget — Require

HOD approval above 50k'

 ELSE 'Low Budget — Require Dean approval for any purchase'

 END;

 -- Manager message using inline CASE

 v_manager_msg := CASE

 WHEN v_manager IS NULL THEN 'No Manager Assigned'

 ELSE 'Has Manager'

 END;

 -- Print results

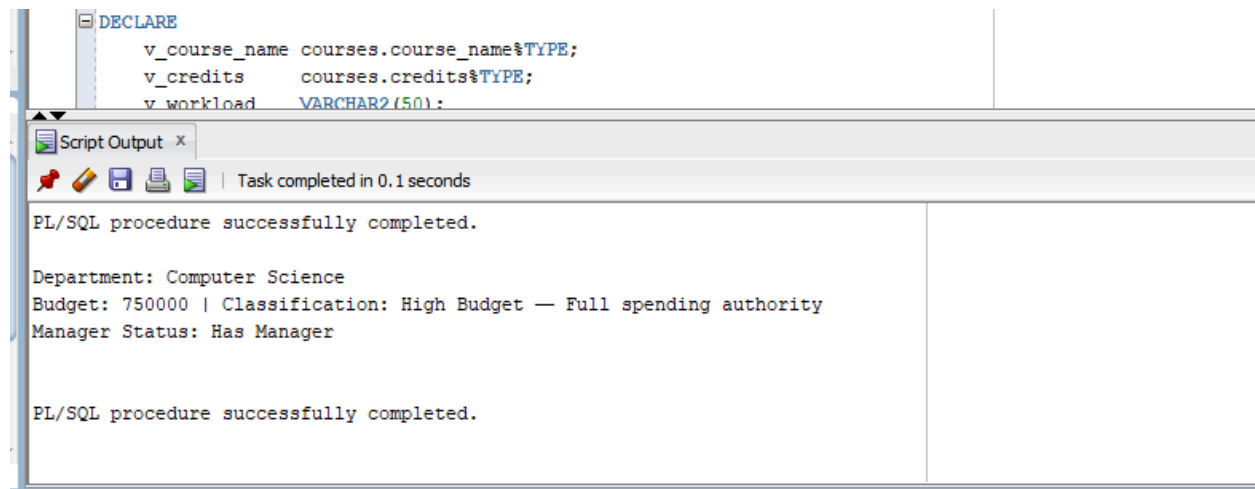
 DBMS_OUTPUT.PUT_LINE('Department: ' || v_dept_name);

 DBMS_OUTPUT.PUT_LINE('Budget: ' || v_budget || ' | Classification: ' || v_class);

```

    DBMS_OUTPUT.PUT_LINE('Manager Status: ' || v_manager_msg);
END;
/

```



```

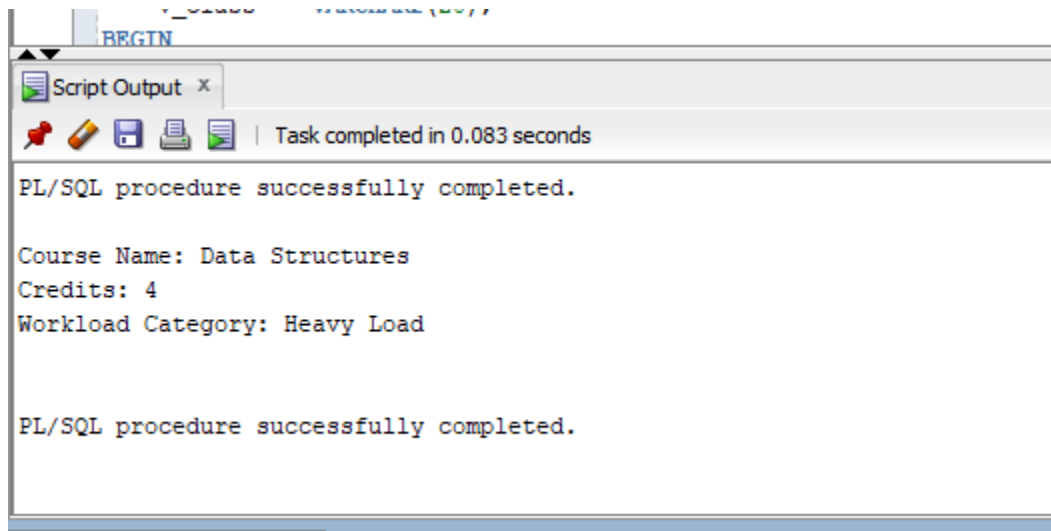
-- Task 3.3 — Course Credit Load
SET SERVEROUTPUT ON
DECLARE
    v_course_name courses.course_name%TYPE;
    v_credits      courses.credits%TYPE;
    v_workload     VARCHAR2(50);
BEGIN
    -- Fetch course info
    SELECT course_name, credits
    INTO v_course_name, v_credits
    FROM courses
    WHERE course_id = 104;

    -- Determine workload using CASE
    v_workload := CASE
        WHEN v_credits BETWEEN 1 AND 2 THEN 'Light Load'
        WHEN v_credits = 3 THEN 'Standard Load'
        WHEN v_credits = 4 THEN 'Heavy Load'
        WHEN v_credits = 5 THEN 'Overload — Requires special permission'
        ELSE 'Unknown Load'
    END;

    -- Print results
    DBMS_OUTPUT.PUT_LINE('Course Name: ' || v_course_name);
    DBMS_OUTPUT.PUT_LINE('Credits: ' || v_credits);
    DBMS_OUTPUT.PUT_LINE('Workload Category: ' || v_workload);
END;

```

/



```
-- =====  
-- QUESTION 4 — Loops  
-- Task 4.1 — GPA Improvement Tracker  
-- Student: Kamagaju Anitha | ID: 28608  
-- =====
```

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_gpa    NUMBER(3,2) := 2.0; -- starting GPA
```

```
    v_semester NUMBER := 1;
```

```
    v_class   VARCHAR2(20);
```

```
BEGIN
```

```
    LOOP
```

```
        -- Determine GPA classification using CASE
```

```
        v_class := CASE
```

```
            WHEN v_gpa >= 3.7 AND v_gpa <= 4.0 THEN 'Summa Cum Laude'
```

```
            WHEN v_gpa >= 3.3 AND v_gpa <= 3.6 THEN 'Magna Cum Laude'
```

```
            WHEN v_gpa >= 3.0 AND v_gpa <= 3.2 THEN 'Cum Laude'
```

```
            WHEN v_gpa >= 2.0 AND v_gpa <= 2.9 THEN 'Satisfactory'
```

```
            ELSE 'Academic Probation'
```

```
        END;
```

```
        -- Print semester and GPA
```

```
        DBMS_OUTPUT.PUT_LINE('Semester ' || v_semester ||
```

```

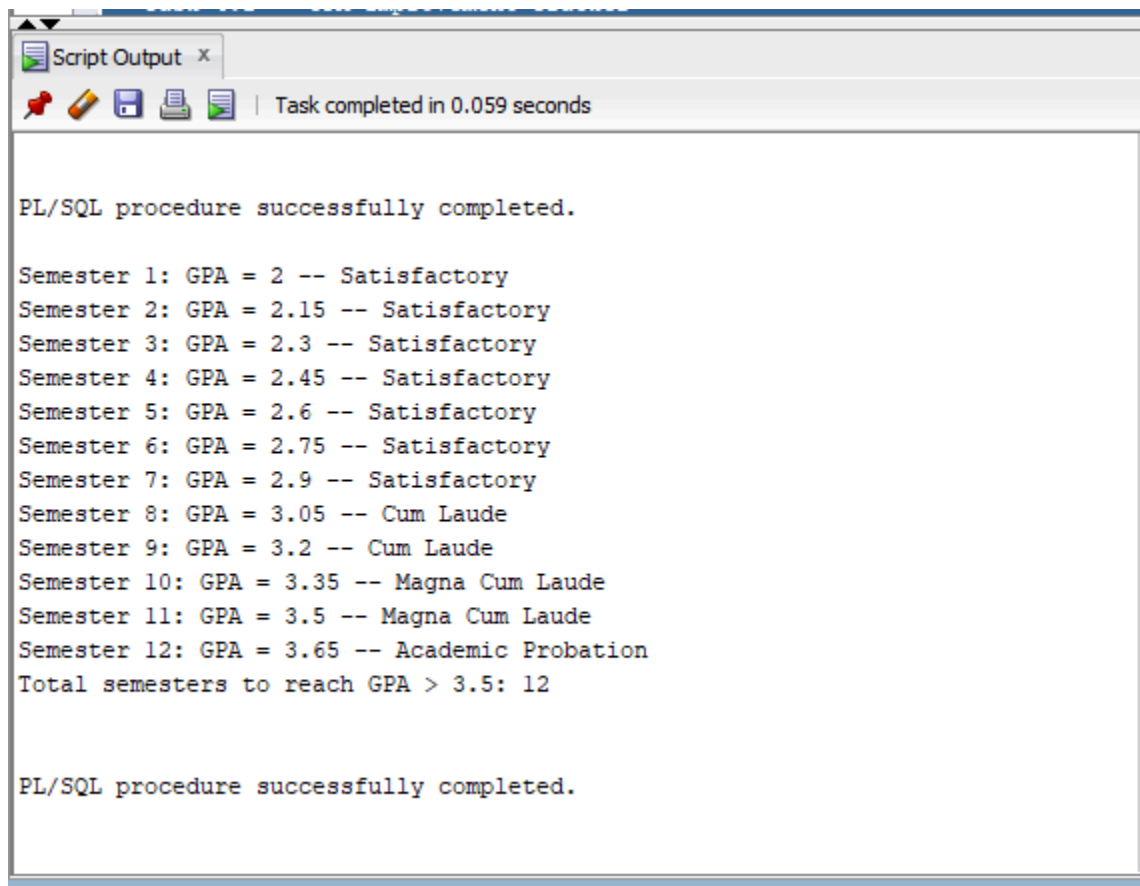
        ': GPA = ' || v_gpa ||
        ' -- ' || v_class);

    -- Exit if GPA exceeds 3.5
    EXIT WHEN v_gpa > 3.5;

    -- Increase GPA and semester counter
    v_gpa := v_gpa + 0.15;
    v_semester := v_semester + 1;
END LOOP;

-- Print total semesters
DBMS_OUTPUT.PUT_LINE('Total semesters to reach GPA > 3.5: ' || v_semester);
END;
/

```



```

PL/SQL procedure successfully completed.

Semester 1: GPA = 2 -- Satisfactory
Semester 2: GPA = 2.15 -- Satisfactory
Semester 3: GPA = 2.3 -- Satisfactory
Semester 4: GPA = 2.45 -- Satisfactory
Semester 5: GPA = 2.6 -- Satisfactory
Semester 6: GPA = 2.75 -- Satisfactory
Semester 7: GPA = 2.9 -- Satisfactory
Semester 8: GPA = 3.05 -- Cum Laude
Semester 9: GPA = 3.2 -- Cum Laude
Semester 10: GPA = 3.35 -- Magna Cum Laude
Semester 11: GPA = 3.5 -- Magna Cum Laude
Semester 12: GPA = 3.65 -- Academic Probation
Total semesters to reach GPA > 3.5: 12

PL/SQL procedure successfully completed.

```

```

-- Task 4.2 — WHILE Loop: Student Score Analyzer
SET SERVEROUTPUT ON
DECLARE
    CURSOR c_scores IS
        SELECT score
        FROM enrollments

```

```
WHERE course_id = 101
AND score IS NOT NULL;
```

```
v_score      enrollments.score%TYPE;
v_total      NUMBER := 0;
v_count      NUMBER := 0;
v_highest    NUMBER := NULL;
v_lowest     NUMBER := NULL;
v_distinctions NUMBER := 0;
BEGIN
  OPEN c_scores;
  LOOP
    FETCH c_scores INTO v_score;
    EXIT WHEN c_scores%NOTFOUND;

    -- Increment counters and totals
    v_count := v_count + 1;
    v_total := v_total + v_score;

    -- Track highest and lowest
    IF v_highest IS NULL OR v_score > v_highest THEN
      v_highest := v_score;
    END IF;

    IF v_lowest IS NULL OR v_score < v_lowest THEN
      v_lowest := v_score;
    END IF;

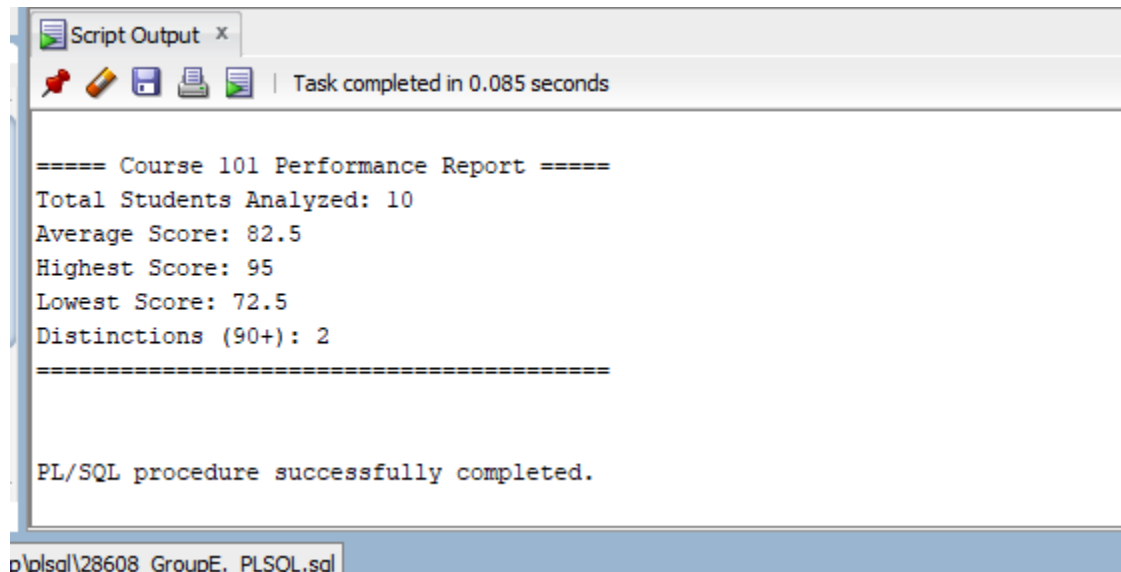
    -- Count distinctions (score >= 90)
    IF v_score >= 90 THEN
      v_distinctions := v_distinctions + 1;
    END IF;
  END LOOP;
  CLOSE c_scores;

  -- Print summary report
  DBMS_OUTPUT.PUT_LINE('==== Course 101 Performance Report ====');
  DBMS_OUTPUT.PUT_LINE('Total Students Analyzed: ' || v_count);
  IF v_count > 0 THEN
    DBMS_OUTPUT.PUT_LINE('Average Score: ' || ROUND(v_total/v_count, 2));
  ELSE
    DBMS_OUTPUT.PUT_LINE('Average Score: N/A');
  END IF;
  DBMS_OUTPUT.PUT_LINE('Highest Score: ' || NVL(TO_CHAR(v_highest), 'N/A'));
```

```

DBMS_OUTPUT.PUT_LINE('Lowest Score: ' || NVL(TO_CHAR(v_lowest), 'N/A'));
DBMS_OUTPUT.PUT_LINE('Distinctions (90+): ' || v_distinctions);
DBMS_OUTPUT.PUT_LINE('=====');
END;
/

```



The screenshot shows a 'Script Output' window with a toolbar at the top. The status bar indicates 'Task completed in 0.085 seconds'. The output text is as follows:

```

===== Course 101 Performance Report =====
Total Students Analyzed: 10
Average Score: 82.5
Highest Score: 95
Lowest Score: 72.5
Distinctions (90+): 2
=====

PL/SQL procedure successfully completed.

```

At the bottom of the window, the file path 'p:\plsql\28608 GroupE. PLSQL.sql' is visible.

```

-- QUESTION 4 — Loops
-- Task 4.3 — Nested FOR Loops: Enrollment Matrix
-- =====
SET SERVEROUTPUT ON
DECLARE
    v_grade      enrollments.grade%TYPE;
    v_enroll_count NUMBER := 0;
BEGIN
    -- Outer loop: first 5 students
    FOR s_id IN 1001..1005 LOOP
        -- Inner loop: first 5 courses
        FOR c_id IN 101..105 LOOP
            BEGIN
                -- Try to fetch enrollment and grade
                SELECT grade
                INTO v_grade
                FROM enrollments
                WHERE student_id = s_id
                AND course_id = c_id;

                -- Enrollment exists
            
```

```

        DBMS_OUTPUT.PUT_LINE('Student ' || s_id || ' | Course ' || c_id || ' | ENROLLED |
Grade: ' || NVL(v_grade, '-')');
        v_enroll_count := v_enroll_count + 1;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            -- Not enrolled
            DBMS_OUTPUT.PUT_LINE('Student ' || s_id || ' | Course ' || c_id || ' | NOT
ENROLLED');
        END;
    END LOOP;
END LOOP;

-- Print total enrollments
DBMS_OUTPUT.PUT_LINE('-----');
DBMS_OUTPUT.PUT_LINE('Total Enrollments in 5x5 matrix: ' || v_enroll_count);
END;
/

```

```
DBMS_OUTPUT.PUT_LINE('Student | ' || s_id || ' | Course | ' || c_id || ' | Grade | ' || g || ' | Status | ' || s || ' |');

PL/SQL procedure successfully completed.

Student 1001 | Course 101 | ENROLLED | Grade: A
Student 1001 | Course 102 | ENROLLED | Grade: A
Student 1001 | Course 103 | ENROLLED | Grade: -
Student 1001 | Course 104 | NOT ENROLLED
Student 1001 | Course 105 | NOT ENROLLED
Student 1002 | Course 101 | ENROLLED | Grade: B
Student 1002 | Course 102 | NOT ENROLLED
Student 1002 | Course 103 | NOT ENROLLED
Student 1002 | Course 104 | ENROLLED | Grade: B
Student 1002 | Course 105 | NOT ENROLLED
Student 1003 | Course 101 | NOT ENROLLED
Student 1003 | Course 102 | ENROLLED | Grade: A
Student 1003 | Course 103 | ENROLLED | Grade: -
Student 1003 | Course 104 | NOT ENROLLED
Student 1003 | Course 105 | NOT ENROLLED
Student 1004 | Course 101 | NOT ENROLLED
Student 1004 | Course 102 | NOT ENROLLED
Student 1004 | Course 103 | ENROLLED | Grade: C
Student 1004 | Course 104 | NOT ENROLLED
Student 1004 | Course 105 | ENROLLED | Grade: B
Student 1005 | Course 101 | ENROLLED | Grade: A
Student 1005 | Course 102 | ENROLLED | Grade: A
Student 1005 | Course 103 | NOT ENROLLED
Student 1005 | Course 104 | NOT ENROLLED
Student 1005 | Course 105 | NOT ENROLLED
-----
Total Enrollments in 5x5 matrix: 11

PL/SQL procedure successfully completed.
```

```
-- =====
-- QUESTION 5 — Combined Challenge
-- Semester Performance Report
-- Student: Kamagaju Anitha | ID: 28608
-- =====
```

SET SERVEROUTPUT ON;

DECLARE

```
-- Student profile variables
v_student_name  students.first_name%TYPE;
v_student_gpa   students.gpa%TYPE;
v_student_status students.status%TYPE;
```



```
v_enrollment_date students.enrollment_date%TYPE;  
v_years_enrolled NUMBER(5,2);  
v_student_flag VARCHAR2(20);
```

```
-- GPA Standing  
v_gpa_standing VARCHAR2(50);  
v_scholarship_note VARCHAR2(100);
```

```
-- Course summary  
CURSOR c_courses IS  
    SELECT e.course_id, e.grade, e.score, c.course_name, c.credits  
    FROM enrollments e  
    JOIN courses c ON e.course_id = c.course_id  
    WHERE e.student_id = 1007;
```

```
v_total_courses NUMBER := 0;  
v_total_credits NUMBER := 0;  
v_best_course VARCHAR2(100);  
v_best_score NUMBER := -1;  
v_courses_in_progress NUMBER := 0;
```

```
BEGIN
```

```
-- Step 1: Student Profile  
SELECT first_name || ' ' || last_name, gpa, status, enrollment_date  
INTO v_student_name, v_student_gpa, v_student_status, v_enrollment_date  
FROM students  
WHERE student_id = 1007;
```

```
-- Compute years enrolled  
v_years_enrolled := MONTHS_BETWEEN(SYSDATE, v_enrollment_date)/12;
```

```
-- Flag new/continuing  
IF v_years_enrolled > 1 THEN  
    v_student_flag := 'Continuing Student';  
ELSE  
    v_student_flag := 'New Student';  
END IF;
```

```
-- Step 2: GPA Standing  
v_gpa_standing := CASE  
    WHEN v_student_gpa >= 3.7 THEN 'Summa Cum Laude'  
    WHEN v_student_gpa >= 3.3 THEN 'Magna Cum Laude'  
    WHEN v_student_gpa >= 3.0 THEN 'Cum Laude'  
    WHEN v_student_gpa >= 2.0 THEN 'Satisfactory'
```

```

        ELSE 'Academic Probation'
    END;

    IF v_student_gpa >= 3.5 THEN
        v_scholarship_note := 'May qualify for merit scholarship';
    ELSE
        v_scholarship_note := 'N/A';
    END IF;

    -- Step 3: Course Enrollment Summary
    FOR rec IN c_courses LOOP
        v_total_courses := v_total_courses + 1;
        v_total_credits := v_total_credits + rec.credits;

        IF rec.score IS NULL THEN
            v_courses_in_progress := v_courses_in_progress + 1;
        ELSE
            IF rec.score > v_best_score THEN
                v_best_score := rec.score;
                v_best_course := rec.course_name;
            END IF;
        END IF;
    END LOOP;

    -- Step 4: Print Final Report
    DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('SEMESTER PERFORMANCE REPORT');
    DBMS_OUTPUT.PUT_LINE('AUCA — DB Development with PL/SQL');
    DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('Student: ' || v_student_name || ' (ID: 1007)');
    DBMS_OUTPUT.PUT_LINE('Status: ' || v_student_status || ' | ' || v_student_flag);
    DBMS_OUTPUT.PUT_LINE('Current GPA: ' || v_student_gpa || ' | Standing: ' ||
v_gpa_standing);
    DBMS_OUTPUT.PUT_LINE('Scholarship Note: ' || v_scholarship_note);
    DBMS_OUTPUT.PUT_LINE('-----');
    DBMS_OUTPUT.PUT_LINE('Courses Enrolled: ' || v_total_courses);
    DBMS_OUTPUT.PUT_LINE('Total Credits: ' || v_total_credits);
    DBMS_OUTPUT.PUT_LINE('Best Course: ' || NVL(v_best_course,'N/A') ||
        ' (Score: ' || NVL(TO_CHAR(v_best_score),'N/A') || ')');
    DBMS_OUTPUT.PUT_LINE('Courses In Progress: ' || v_courses_in_progress);
    DBMS_OUTPUT.PUT_LINE('=====');

END;
/

```

```
Script Output x
Task completed in 0.096 seconds

PL/SQL procedure successfully completed.

=====
SEMESTER PERFORMANCE REPORT
AUCA — DB Development with PL/SQL
=====
Student: Grace Wilson (ID: 1007)
Status: Active | Continuing Student
Current GPA: 3.7 | Standing: Summa Cum Laude
Scholarship Note: May qualify for merit scholarship
-----
Courses Enrolled: 3
Total Credits: 10
Best Course: Calculus I (Score: 91)
Courses In Progress: 1
=====

PL/SQL procedure successfully completed.
```